ACTIVIDAD

Dosier de Actividades del Tema: Control de excepciones

Desarrollo de Aplicaciones Web/ Desarrollo de Aplicaciones Multiplataforma/Administración de Sistemas Informáticos y Redes

Programación

Linkia FP

Se debe entregar un zip que contenga todos los archivos. java que has creado. Para poder aprobar un ejercicio, éste debe poder ejecutarse sin errores.

Gestión de una tienda de libros con excepciones personalizadas y propagación de excepciones.

Objetivo: Practicar la gestión de una tienda de libros en Java, creando excepciones personalizadas y utilizando la propagación de excepciones para manejar situaciones excepcionales.

EJERCICIO 1: Implementar la gestión de la tienda de libros:

- Crea una clase principal que represente la tienda de libros.
- Atributos: String titulo, int stock;

```java
 *
 * @author cristiancantero
 */
public class Libro {
    private String titulo;
    private int stock;


    public Libro() {
    }

    public Libro(String titulo, int stock) {
        this.titulo = titulo;
        this.stock = stock;
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }

    @Override
    public String toString() {
        return "Libro: " + "Titulo= " + titulo + " | " + " stock= " + stock;
    }
```


Linkia FP

- En la clase principal, la que contiene el método main(), implementa métodos para agregar libros al inventario, realizar ventas y buscar libros. ***Prefiero crear una clase para gestionar las funciones. Se llama Funciones y tiene todas las funciones que el main va a llamar. Prefiero hacerlo así por una cuestión de separación de responsabilidades.***
    - agregarLibro(String titulo, int stock) ***agregaré HashMap para guardar el stock como atributo de entorno.***

```
 * @author cristiancantero
 */
public class Funciones {

    private HashMap<String, Libro> listaLibros = new HashMap<>();

 public void agregarLibro(String titulo, int stock) throws ExcepcionExiste{
     titulo = titulo.toLowerCase(); //para q no me pillen mays y minusc
     if(listaLibros.containsKey(titulo)){
         throw new ExcepcionExiste("Este libro ya existe en el stock");
     } else {
         Libro libro = new Libro(titulo, stock);
         listaLibros.put(titulo,libro);
         System.out.println("Libro añadido correctamente");
     }

    }
```

Como se puede ver lo primero como atributo de entorno está declarado el HashMap listaLibros. La funcion agregarLibro() recibe como parametros un String con el título y un int que representará el stock. Seguidamente en la firma uso throws con la excepción de cosecha propia ExcepcionExiste que nos libra de tener que hacer try-catch para recoger el programa cuando explota. Seguidamente le indico a la variable titulo que usará el método toLowerCase() que hace que java obvie mayúsculas y minúsculas. Esto se hace por si el usuario no introdujera este tipo de letras aun cuando java las hubiese almacenado así. Seguimos con el condicional. Si dentro de listaLibros encontramos titulo usando el método propio de HashMap .containsKey que nos indica si hay algún elemento coincidente con titulo. Si fuera así se lanza la excepcion de cosecha propia con un mensaje. Si no creamos una instancia de libro con el título y el stock que hemos recibido desde el main y lo guardamos en el hashmap usando el metodo .put. Acabamos con un mensaje de proceso terminado correctamente.

- venderLibro(String titulo) throws ExcepcionLibroNoEncontrado, ExcepcionStockAgotado { … }

```
public void venderLibro(String titulo)throws ExcepcionNullLibro, ExcepcionStockAgotado{
    Libro libro = listaLibros.get(titulo);
    if (libro == null){
        throw new ExcepcionNullLibro("El libro no existe");
    }
    if(libro.getStock()==0){
        throw new ExcepcionStockAgotado("El stock está agotado");
    }

    libro.setStock(libro.getStock()-1);
    System.out.println("Libro vendido correctamente");
}
```

Esta función recibe como parámentro solamente el título. En la firma he incluido dos excepciones de cosecha propia. Creamos una instancia de libro y obtenemos el título a través del método get. Usamos el condicional if para decir que si libro == null (esto es así porque el título del libro es un String y se tiene que gestionar con null) lancemos la excepcion que confirma que ese libro no esta en el stock. Luego comprobamos el stock y accedemos a el a través de get (recuerda que stock está en la clase Libro). Si el stock es 0 el libro está agotado. Si no saltaron las excepciones seteamos el stock para refrescarlo restandole 1. Y acabamos con un mensaje de vendido correctamente.

- Libro buscarLibro(String titulo) throws ExcepcionLibroNoEncontrado { … }

```
public Libro buscarLibro(String titulo) throws ExcepcionNullLibro{
    titulo = titulo.toLowerCase();
    Libro libro = listaLibros.get(titulo);
        if(libro == null){
            throw new ExcepcionNullLibro("El libro no existe");
        }
        return libro;
}
```

Esta

- Utiliza bloques try-catch para tratar las excepciones personalizadas lanzadas por los métodos. 😱😱😱 he visto esto tan tarde que te lo arreglo sobre la marcha. Dont worry. Voy hacer el try/catch sobre el stock en el main porque así a bote pronto no se me ocurre que podria hacer, las clases están bien arregladas. … Hay que pensar que el título puede llevar mayusculas, minusculas, caracterres, números. Ademas en el main porque es donde voy a introducir los datos. De veras que le he dado vueltas y no es que no sepa como se hace, es que no se porque se tiene que hacer (los try/catch sobre las funciones). Tengo las funciones definidas de tal manera que no hace falta. Se que el enunciado me pide que haga esto en el main pero es que es un 'toc' lo siento. Necesito separar responsabilidades. No puedo modelar funciones en el main.. puro toc… seguro que estoy equivocado. En fin.

  El bloque del outpout nos sacará de dudas.

EJERCICIO 2: Crear excepciones personalizadas:

- Crea al menos dos clases de excepciones personalizadas relacionadas con la gestión de una tienda de libros. Por ejemplo:
  - ExcepcionLibroNoEncontrado: una excepción que se lanza cuando un libro solicitado no está disponible en la tienda.
  - ExcepcionStockAgotado: una excepción que se lanza cuando el stock de un libro se agota y no se puede realizar una venta.

```java
package ACT11;

/**
 *
 * @author cristiancantero
 */


    public class ExcepcionStockAgotado extends Exception{
        public ExcepcionStockAgotado (String mensaje){
            super(mensaje);
        }
    }
```

Aquí he hecho una cosa rara… (falta de control y *conocimiento le llamo yo). Hice la excepción del stock agotado, del null y de la falta de existencias…. Yo creo que podría haberlo dejado todo en el null pero como ya lo tenía hecho y no hacía daño decidí dejarlo.*

```java
package ACT11;

/**
 *
 * @author cristiancantero
 */
public class ExcepcionNullLibro extends Exception{

        public ExcepcionNullLibro (String mensaje){
            super(mensaje);
        }
    }
```

Este código no tiene ningún misterio. De hecho es una plantilla sacada de los imprimibles de Linkia. Todas mis excepciones han sido estas tres líneas.

EJERCICIO 3: Ejecutar el código y gestionar las excepciones:

- Crea un menú donde el usuario pueda
    1. Agregar libro:
    2. Vender libro: Si no hay stock suficiente o no existe el libro muestra un mensaje a partir de la excepción personalizada.
    3. Buscar libro: Si no existe el libro muestra un mensaje a partir de la excepción personalizada.
    4. Salir
- Captura y trata adecuadamente las excepciones en bloques catch correspondientes.
- Proporciona mensajes de error descriptivos y acciones correctivas al usuario.

```
package ACT11;

import java.util.Scanner;
/**
 *
 * @author cristiancantero
 */
public class Main {
```


Linkia FP

```
public static void main(String[] arg) {
    Scanner sc = new Scanner(System.in);
    Funciones funciones = new Funciones();

    int pocion; // xD

    do {
        System.out.println("1. Agregar libro");
        System.out.println("2. Vender libro");
        System.out.println("3. Bujcar libro"); //xD
        System.out.println("0. Salir dew"); //xD

        pocion = sc.nextInt();
        sc.nextLine();
        try {
            switch (pocion) {
                case 1: {
                    System.out.println("Introduce el nombre del libro");
                    String nombreLibro = sc.nextLine();
                    System.out.println("Introduce el stock: ");
                    int stock = sc.nextInt();
                    sc.nextLine();
                    funciones.agregarLibro(nombreLibro, stock);
                }
                break;
                case 2: {
                    System.out.println("Introduce el nombre del libro");
                    String nombreLibro = sc.nextLine();
                    funciones.venderLibro(nombreLibro);
                }
                break;
                case 3: {
                    System.out.println("Introduce el nombre del libro");
                    String nombreLibro = sc.nextLine();
                    Libro libro = funciones.buscarLibro(nombreLibro);
                    System.out.println(libro);

                }

                break;
                case 0:
```

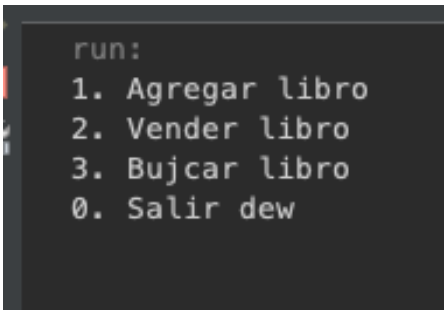

Linkia FP

```
                System.out.println("Programa cerrado correctamente");
                return;
            default:
                System.out.println("Introduce un caracter válido");

            }
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    } while (pocion != 0);

  }
}
```

Los ménus que venimos haciendo desde el principio del módulo.

BLOQUE DEL OUTPUT

Run file…



Linkia FP

```
run:
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
1
Introduce el nombre del libro
Quijote
Introduce el stock:
3
Libro añadido correctamente
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
1
Introduce el nombre del libro
Juego de tornos
Introduce el stock:
76
Libro añadido correctamente
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
2
Introduce el nombre del libro
Quijote
El libro no existe 😳
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
```

Opción 1 agregar libro, parece q todo bien.

Volvemos a agregar otro libro…

Y cuando vamos a vender libro…

Pues el libro parece q no existe…

Asi que toca debuggear.

… protocolo==> mirar la entrada de datos de usuario, los .toLowerCase()… y ahora la guinda del pastel. El señor .trim(). La ausencia de este señor hacía que una pequeña "equivocación", un espacio de más o de menos, aunque estuviera el nombre bien escrito… frustraba la inserción del objeto… .trim() lo vi cuando hacía el ejercicio de la lectura y escritura secuencial de archivos.

Asi que lo arreglé de la siguiente manera:

Busqué la funcion venderLibro() y al principio le añadi las intrucciones para que titulo obviase las mays y minus y también los espacios.

titulo = titulo.toLowerCase().trim();

Y asunto arreglado:

```
run:
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
1
Introduce el nombre del libro
quijote
Introduce el stock:
4
Libro añadido correctamente
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
2
Introduce el nombre del libro
quijote
Libro vendido correctamente
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
3
Introduce el nombre del libro
quijote
Libro: Titulo= quijote |  stock= 3
1. Agregar libro
2. Vender libro
3. Bujcar libro
0. Salir dew
```

Opción 1: introduzco el nombre

Y el stock… fíjate que son 4 🤭

Opción 2 buscarLibro:

Le ponemos el nombre y nos dice que está vendido correctamente.

Opción 3 buscar: pongo el nombre y me devuelve el titulo y el stock. Fíjate que ahora hay un libro menos!!! ✌🏽

Linkia FP

CONCLUSIONES: La verdad que montar las exceptions no ha sido complicado. En este caso han sido como 3 clases iguales extra. Y luego llamarlas. Nada más. Aquí el asunto es lo de siempre. El HashMap, el 'chip' a la hora de trabajar con el. (¡No es necesario Iterar simpre, HashMap nos lo pone a huevo en este aspecto!). No voy a decir que lo he aprendido antes de llegar al HashMap no me ha servido de nada porque no sería cierto. Pero la sensación de estar haciendo una cosa nueva y difícil sobre todo porque falta conocimiento y practica… está ahí y pesa. Así que como siempre mi conclusión es la misma solo que ahora le añado un pequeño matiz. ***Hay que sacar tiempo para estudiar, y hay que estudiar mucho. Pero sobre todo: hay que aprender a aprender.*** Creo q es la primera vez que entiendo el 'mantra' de *pensar como un programador*… no porque yo ya haya conseguido pensar como tal, sino por lo que significa. No le veia el horizonte al significado. Ahora creo empezar a entender un poco a que se refiere. Como piensas la resolucion de los problemas, como los abordas con las herramientas q ya conoces… Hay mucha tela que cortar aqui. Así que despacito y a compás. ∫

Bibliografia consultada: Imprimibles Linkia, busquedas Google.

Herramientas de aprendizaje: Chat gpt

Linkia FP

Linkia FP